



## SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

### Descriptive Written Exam

**COURSE NAME: Software Testing**

**COURSE CODE:CSA37**

#### COURSE OUTCOME COVERED

**CO1:** *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

#### ***Code of Conduct:***

*I **Manoj A / 192421295** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

#### ***Signature of the student:***

# Answers

## Question 1

A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

### 1. Introduction

Testing is not a casual activity of "trying the software to see what happens". It is an engineering discipline: it is planned, designed, measured, repeatable and improved over time, just like any other engineering process. In the banking application, failures occur because testers pick inputs at random, do not record what was tested and cannot repeat or measure their work. The redesign therefore treats testing as an engineered process with defined inputs (requirements), activities (design, execution), outputs (reports) and measures (coverage, defect density).

### 2. Redesigned Testing Approach (Testing as an Engineering Activity)

- **Plan:** define scope, objectives, risks, entry/exit criteria, schedule and resources in a Test Plan.
- **Design:** derive test cases systematically from requirements using test design techniques; each case has ID, precondition, input, expected result and traceability to a requirement.
- **Execute:** run the cases in a controlled test environment and record actual results.
- **Report and track:** log every defect with severity, priority and steps to reproduce.
- **Measure and improve:** collect metrics (coverage, pass rate, defect density, defect removal efficiency) and feed lessons back into the process.

### 3. Introducing Systematic Test Design, Tools and Standards

- **Systematic test design:** Requirement Traceability Matrix (RTM), equivalence partitioning, boundary value analysis, decision tables and state-transition testing replace guesswork.

- **Tools:** Test management (Jira/Zephyr, TestRail), defect tracking (Jira, Bugzilla), automation (Selenium, Postman/REST Assured for APIs), performance (JMeter), and CI tools (Jenkins) for regression on every build.
- **Standards:** IEEE 829 for test documentation, ISO/IEC/IEEE 29119 for software testing processes, ISTQB terminology, and domain standards such as PCI-DSS for card data security in banking.
- **Governance:** peer review of test cases, a naming convention, mandatory defect template and exit criteria before release.

Because every requirement is mapped to test cases, missing functionality and boundary errors are found before release, which reduces production failures and increases customer trust.

#### 4. Techniques to Replace Ad-hoc Testing

- Equivalence Partitioning and Boundary Value Analysis for amounts, limits and dates.
- Decision table testing for business rules (e.g., transfer allowed or rejected).
- State-transition testing for account states (Active, Dormant, Blocked).
- Risk-based testing to give more effort to money-moving features.
- Experience-based techniques such as structured exploratory testing (with charters and time-boxes) and error guessing, used to supplement, not replace, design-based tests.

#### 5. Practical Example: Fund Transfer Feature

Requirement: a customer can transfer Rs. 1 to Rs. 50,000 per transaction and the balance must not go negative. Ad-hoc testers might only try Rs. 1,000 and see success. The engineered approach produces:

Defects such as acceptance of Rs. 50,001 or a negative balance, which random testing would probably miss, are exposed by these cases and each is tracked and re-tested after the fix.

#### 6. Conclusion

Treating testing as an engineering activity, with systematic design, tools and standards, converts the banking team's unpredictable testing into a measurable, repeatable process that finds more defects earlier and improves overall software quality.

## Question 2

An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

### 1. Introduction

A defined testing process ensures that every release is tested in the same planned way. In the ATM system, results are inconsistent because testing activities differ from build to build. The solution is to embed testing in every SDLC phase (the V-model view), so that verification and validation happen continuously.

### 2. Stages of the Test Process and Their Relevance

- **Test planning and control:** defines objectives, scope, resources and exit criteria; control compares progress with plan.
- **Test analysis and design:** converts requirements into test conditions and test cases.
- **Test implementation:** prepares test data (cards, PINs, balances), environment and scripts.
- **Test execution:** runs cases, compares actual and expected results, logs defects.
- **Evaluating exit criteria and reporting:** decides whether testing is complete using metrics.
- **Test closure:** archives test-ware and records lessons learned for future releases.

### 4. Application to ATM Withdrawal Functionality

Requirement: the user inserts a valid card, enters the correct PIN, requests an amount in multiples of Rs. 100 within the daily limit and available balance; cash is dispensed and the balance is updated.

Unit tests cover the amount-validation logic, integration tests cover the ATM-to-bank message flow, system tests cover end-to-end scenarios and regression tests re-run all of the above after every fix.

## **5. How Process Consistency Reduces Defects**

- The same documented steps are followed every release, so nothing is skipped.
- Early reviews remove defects when fixing them costs the least.
- Traceability ensures each requirement has tests, reducing missed functionality.
- Standard templates for defects and results make outcomes comparable between builds.
- Metrics reveal weak areas so effort can be corrected.

## **6. Expected Outcomes**

- Consistent, repeatable ATM test results across releases.
- Fewer defects escaping to production, especially in cash-handling and transaction rollback.
- Reduced rework cost and faster release cycles.
- Higher reliability, security and customer confidence.

### Question 3

A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

#### 1. Basic Definitions

- **Error (mistake):** a human action that produces an incorrect result, e.g., a developer misunderstands the requirement.
- **Defect (bug/fault):** a flaw in the code or document that results from an error; it exists in the software.
- **Failure:** the observable deviation of the software from expected behaviour when a defect is executed.

Relationship: Error leads to Defect, and Defect (when executed) leads to Failure. Not every defect causes a failure; it depends on whether the faulty code is executed under the right conditions.

#### 2. Illustrating the Relationship

Example: the specification says "Lock the account after 3 wrong attempts". The developer mistakenly writes the counter check as attempts > 3 (error). The code therefore has a defect: the loop allows four attempts. During testing, the tester enters a wrong password four times and the account is still not locked after the third attempt, which is the failure. The same defect would stay hidden if the tester only tried two wrong attempts, showing that a defect may not always produce a failure.

#### 3. Effect of Improper Understanding of the Terms

- Calling every problem a "bug" hides whether the cause is a requirement error, coding error or environment issue, so root cause analysis becomes weak.
- Failures reported without steps and observed result are treated as defects in code, wasting developer time.
- Testers may stop after observing a failure and not look for related defects.

- Management misreads metrics, e.g., counting failures as defects inflates defect counts.
- Disputes between testers and developers arise because the same issue is described inconsistently.

#### **4. Improving Defect Reporting Clarity**

- Use a standard template: Defect ID, title, module, environment, build version, severity, priority, steps to reproduce, expected result, actual result, attachments (screenshots/logs), status and reporter.
- Write a short, specific title, e.g., "Login fails for valid email with uppercase letters" instead of "Login not working".
- Describe the failure (observed behaviour) separately from the suspected cause.
- Make steps reproducible and attach evidence.
- Review reports and use the defect life cycle (New, Assigned, Fixed, Retest, Closed).

Good report: "Steps: 1. Open login page. 2. Enter Manoj@mail.com and valid password. Expected: user logs in. Actual: 'Invalid username' message. Severity: High." This is clear enough for a developer to find and fix the defect quickly.

#### **5. Conclusion**

Clear use of the terms error, defect and failure helps testers describe problems correctly, find root causes and communicate effectively, which improves the quality of the web application.

## Question 4

A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

### 1. Introduction

The observation matches the testing principle of defect clustering: a small number of modules usually contain most of the defects (the Pareto 80/20 rule). The team should use this and the other seven testing principles to plan tests.

### 2. Applying Defect Clustering

- Analyse defect data from previous releases and cycles to find modules with the highest defect density.
- Identify causes such as complex logic, frequent changes, inexperienced developers or unclear requirements.
- Increase test depth, code reviews and regression in those modules and plan extra time for them.
- Monitor after fixes, since clusters can move to other modules.

### 3. Prioritising Test Cases Effectively

- **Risk-based:** rank by probability of failure times impact.
- **Defect-history based:** modules with more past defects get tests first.
- **Requirement/business priority:** core features before cosmetic ones.
- **Change-based:** recently modified code is tested earlier.
- **Coverage-based:** tests that cover the most requirements run first.

### 4. Efficiency Improvements

- Effort is focused where defects are most likely, so more defects are found in less time.
- Delays are reduced since critical modules are stable before release.
- Test cases are not wasted on low-risk areas.

- Regular updating avoids the pesticide paradox and keeps tests effective.

### **5. Real-Time Example**

In an online shopping project, defect logs show that the Payment module has 60% of all defects. The test manager assigns the most experienced testers to Payment, adds boundary and integration tests with the bank gateway, performs additional code reviews and runs regression daily. Low-risk modules such as Profile receive only basic tests. As a result, critical payment defects are found two weeks earlier and the release delay is avoided.

## Question 5

A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

### 1. Introduction

A defect repository is a central store of all defect information (description, type, severity, phase found, root cause, status, fix). If it is only used for logging, valuable knowledge is wasted and the same defects return. Using classification and repository analysis turns it into a learning tool.

### 2. How Defects Originate

- **Requirements phase:** ambiguous, missing or conflicting requirements.
- **Design phase:** wrong architecture, poor interface or algorithm design.
- **Coding phase:** logic mistakes, wrong data types, missing validations, poor exception handling.
- **Testing phase:** incorrect test data or environment setup.
- **Maintenance:** changes and patches introduce new defects (regression).
- Underlying causes include time pressure, lack of skills, poor communication and weak reviews.

### 3. Using Repository Data for Test Design and Prevention

- Identify the most frequent defect types, e.g., boundary errors, and design extra boundary value tests.
- Build a regression suite from defects that have occurred before.
- Find high-defect modules for risk-based test planning.
- Analyse root causes to create checklists for reviews and coding standards.
- Calculate metrics such as defect density, defect removal efficiency and reopen rate.

Example: if the repository shows 15 of 40 defects are "missing input validation" in forms, the team adds validation checklists to code review and designs equivalence and boundary tests for every input field in the next release.

#### **4. Improvements to Avoid Recurring Defects**

- Make fields mandatory: type, root cause, phase injected, module and fix details.
- Hold periodic defect analysis meetings and causal analysis (Pareto charts, fishbone diagrams).
- Search the repository for similar defects before closing, to link duplicates and recurring issues.
- Add every escaped defect to the automated regression suite.
- Feed lessons learned back into requirements reviews, coding guidelines and training.

#### **5. Expected Quality Improvements**

- Reduced repeat defects and reopen rate.
- Higher defect removal efficiency before release.
- Better test coverage of defect-prone areas.
- Lower rework cost and faster, more predictable releases.

## Question 6

A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

### 1. Introduction

Crashes during fund transfer are critical failures because they affect money and customer trust. Applying testing principles such as early testing, defect clustering, exhaustive testing is impossible (so use risk-based selection) and context dependence helps locate and remove the root cause systematically.

### 2. Identifying the Root Cause

- **Reproduce:** record device, OS version, app build, network type and exact steps; check whether the crash is consistent or intermittent.
- **Collect evidence:** crash logs, stack traces (Logcat/Xcode/Firebase Crashlytics), server logs and API responses.
- **Isolate:** determine whether the fault is in the UI, the business logic, the API or the database by testing each layer separately.
- **Vary conditions:** test different amounts, payees, networks (4G, Wi-Fi, weak signal), devices and memory states.
- **Apply defect clustering:** check whether crashes concentrate in one module such as payee validation or OTP handling.
- **Confirm:** after the developer finds the cause, re-test and run regression.

### 3. How Systematic Testing Reduces Such Issues

- Test cases derived from requirements with boundary values, invalid inputs and state transitions cover the cases that random use would miss.
- Early review and unit tests catch null-pointer and validation faults before integration.
- Non-functional tests (performance, compatibility, interruption) expose crash conditions.
- Regression automation prevents fixed crashes from returning.

#### **4. Illustrative Example**

Test: transfer Rs. 5,000 to a new payee on Android 9 with weak network. Step 3 (OTP verification) throws a timeout; the app has no exception handler and closes. Root cause: unhandled timeout exception in the OTP API call. Fix: add timeout handling with retry and a friendly message. Added tests: OTP delay, OTP failure, network drop at each step, and double-tap on Confirm.

#### **5. Improvements in Testing Practices**

- Add a device and OS compatibility matrix and use device clouds (BrowserStack, AWS Device Farm).
- Include interruption testing (calls, notifications, low battery, app switching).
- Use crash analytics in beta releases and stage rollouts.
- Automate regression for critical flows with Appium.
- Perform performance and security testing of the transfer API (JMeter, OWASP checks).
- Run code reviews and static analysis focusing on exception handling.

## Question 7

A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

### 1. Introduction

Testing and debugging are related but different activities. Testing reveals that a failure exists; debugging finds and fixes the defect that caused it. Confusing them means defects are either not found systematically or not fixed properly.

### 2. Roles in Software Quality

- Testing gives information about quality and risk and measures conformance to requirements.
- Debugging improves quality by removing the cause of failures and, with root cause analysis, helps prevent similar defects.
- Together they form a loop: test, report, debug, retest.

### 3. Effects of Improper Usage

- Developers who "test while debugging" check only the failing scenario and miss related defects.
- Without independent testing, developers' assumptions are repeated and defects stay hidden.
- Testers who try to debug waste time and delay execution of remaining tests.
- Fixes made without clear failure reports often introduce new defects (no regression).
- Defect metrics become unreliable, as undocumented fixes are not tracked.

Example: a tester reports "discount not applied". A developer changes the discount formula only for that input and marks it fixed without testing other coupons. Later, coupons for bulk orders fail because no systematic test or regression was done.

### 4. Structured Approach to Integrate Both

- **Step 1 Test:** execute planned test cases and log failures with reproducible steps.

- **Step 2 Triage:** confirm the defect, assign severity and priority.
- **Step 3 Debug:** developer reproduces, finds the root cause (logging, breakpoints, code tracing), and fixes.
- **Step 4 Unit test and review:** developer adds a test for the fix and a peer reviews the code.
- **Step 5 Retest:** tester confirms the fix using the original steps.
- **Step 6 Regression:** run related tests to ensure no side effects.
- **Step 7 Close and learn:** update the repository with root cause and add prevention actions.

## Question 8

An e-commerce platform shows inconsistent behavior across browsers.

Apply testing principles to design a strategy for cross-browser testing.

Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

### 1. Introduction

Cross-browser testing verifies that a web application behaves and looks the same across browsers, versions, operating systems and devices. Because exhaustive testing of every combination is impossible, a risk-based and data-driven strategy is applied.

### 2. How Defects Originate

- Different rendering engines (Blink, WebKit, Gecko) interpret HTML, CSS and JavaScript differently.
- Use of features not supported in older browsers (e.g., newer CSS properties or JS APIs).
- Vendor-specific prefixes and non-standard code.
- Differences in form handling, cookies, local storage, fonts and default styles.
- Screen size, resolution and touch versus mouse differences.
- Third-party scripts and payment plugins behaving differently.

### 3. Cross-Browser Testing Strategy

- **Define scope:** use analytics to choose the top browsers and versions (e.g., Chrome, Edge, Firefox, Safari) on Windows, macOS, Android and iOS.
- **Prioritise:** test critical flows first: search, add to cart, checkout, payment and login.
- **Design tests:** a compatibility matrix of browser x OS x device against test cases for UI layout, functionality, performance and responsiveness.
- **Execute:** manual exploratory checks plus automated scripts on all selected combinations.
- **Report and retest:** log defects with browser, version and screenshots; run regression after fixes.
- **Maintain:** update the matrix when new browser versions are released.

#### 4. Systematic Testing and Reliability

Following the matrix ensures that each critical feature is verified on every supported environment. Defects such as a checkout button overlapping in Safari or a payment pop-up blocked in Firefox are found before users see them, so behaviour becomes consistent and customer complaints and cart abandonment drop.

#### 5. Tools and Techniques

- **Tools:** Selenium WebDriver / Grid, Playwright, Cypress, BrowserStack, LambdaTest, Sauce Labs, Chrome DevTools device mode.
- **Techniques:** responsive testing, visual regression testing (Percy, AppliTools), HTML/CSS validators, feature detection (Can I Use) and graceful degradation / progressive enhancement.
- **Process:** run automated suites in CI on every build, use parallel execution to shorten time and use analytics to refine the matrix.

## Question 9

A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

### 1. Introduction

Incorrect salary calculation caused by logical errors is a functional/logic class defect. Classifying defects by type, origin, severity and root cause helps the team understand them and plan effective testing.

### 2. Categorising the Payroll Defects

Using the orthogonal view, most of these are logic defects introduced in the coding phase, with some originating from requirements.

### 3. How Understanding Defect Classes Improves Testing

- Logic defects point to white-box techniques (statement, branch and condition coverage) and decision tables.
- Boundary defects point to BVA and equivalence partitioning.
- Requirement-origin defects point to requirement reviews and acceptance tests.
- Severity classes guide test priority: tax and net salary tests run first.
- The class counts reveal trends and which skills or checklists are needed.

### 4. Prevention in Future Releases

- Write clear business rules (tax slabs, overtime, deductions) and review them with HR and finance.
- Use decision tables to capture all rule combinations and derive test cases from them.
- Peer code review with a logic checklist; unit tests for each calculation function.
- Use decimal/BigDecimal types for currency.
- Add every defect found to an automated regression suite.
- Compare results with an independent oracle such as a spreadsheet calculation.

Example:  $\text{salary} = \text{basic} + \text{HRA} + \text{overtime} - \text{tax} - \text{PF}$ . For basic Rs. 40,000 and 10 overtime hours, the tester computes the expected net pay independently and automates the comparison for a data set of 50 employees.

## **5. Improvements in Testing Strategy**

- Combine black-box (BVA, decision tables) and white-box (branch/condition coverage) testing.
- Perform parallel testing against the previous correct payroll run.
- Use a large data-driven test set covering all grades, slabs and leave cases.
- Mutation testing and code review checklists to judge test strength.
- Regression testing before every release and root-cause analysis in the defect repository.

## Question 10

A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

### 1. Introduction

Executing random tests gives unpredictable coverage and no repeatability. Treating testing as an engineering activity means each test is designed, reviewed, traced to a requirement, executed and measured.

### 2. Redesigned Testing Process

- Analyse requirements and identify test conditions.
- Select test design techniques and create test cases with ID, objective, preconditions, steps, test data, expected result and priority.
- Review test cases and map them to requirements in a traceability matrix.
- Execute in a controlled environment, record actual results and log defects.
- Measure coverage and defect metrics and refine the test set.

### 3. Importance of Structured Test Case Design

- Ensures coverage of all requirements, including negative and boundary conditions.
- Makes testing repeatable, so results can be compared across builds.
- Allows new team members and automation tools to reuse the tests.
- Helps prioritise tests by risk when time is limited.
- Provides evidence of testing for audits and management decisions.
- Finds defects with fewer tests, saving time.

### 4. Illustration of Improved Defect Detection

Requirement: a password must be 8 to 12 characters. A random tester may try "abc123" and "password1". A designed set uses BVA and equivalence classes:

If the developer coded  $\text{length} > 8$  instead of  $\text{length} \geq 8$ , only TC2 reveals the defect. Random testing would probably have missed it.

## **5. Techniques to Enhance Test Effectiveness**

- Black-box: equivalence partitioning, boundary value analysis, decision tables, state-transition and use-case testing.
- White-box: statement, decision and condition coverage.
- Experience-based: error guessing and exploratory testing with charters.
- Risk-based prioritisation, peer reviews of test cases and pairwise testing for many input combinations.
- Regular review of tests to avoid the pesticide paradox.

## Question 11

A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

### 1. Introduction

In a healthcare system, patient data (demographics, prescriptions, billing, lab reports) is shared among modules. Inconsistency, such as a patient's age differing between registration and billing, can harm patients, so data integrity is a top-priority quality attribute.

### 2. Analysis Using Testing Principles

- **Early testing:** review data models and interface specifications first.
- **Defect clustering:** check whether inconsistencies concentrate in interfaces such as Registration to Billing.
- **Exhaustive testing is impossible:** select risk-based data scenarios (critical patient data, allergies, dosage).
- **Context dependence:** healthcare needs strong accuracy, privacy (HIPAA-style rules) and audit trails.
- **Absence-of-errors fallacy:** correct but wrong-for-clinical-use data is still failure.

### 3. How Defects Originate During Development

- Requirements do not define the single source of truth for each data item.
- Design duplicates data in several tables without synchronisation.
- Coding errors: different data types, field lengths or date formats in different modules.
- Interface mismatches in APIs and HL7/FHIR message mapping.
- Missing transaction control, so partial updates are saved.
- Concurrent updates by multiple users and failed data migration.

### 4. Systematic Testing for Data Integrity

Example: a patient's weight is changed from 70 kg to 65 kg in the Nursing module. Test checks that Pharmacy dosage calculation, Billing and the patient summary show 65 kg.

## **5. Improvements in Defect Tracking and Reporting**

- Record module, data item, affected modules and patient-safety impact in each report.
- Use severity levels with the highest rating for patient-safety defects.
- Attach screenshots, database queries and logs.
- Use a tool (Jira/Bugzilla) with workflow and traceability to requirements.
- Hold regular defect triage and track metrics such as defect density per module and time to fix.
- Analyse root causes and add data-consistency tests to the regression suite.

## Question 12

A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue.

Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

### 1. Introduction

Repeated defects indicate that testing is not learning from the past. A defect repository holds the history of all defects and is the main source for analysing why defects recur.

### 2. Analysing the Issue Using the Repository

- Search for duplicate or similar defects across releases to find which ones recur.
- Check whether root causes were recorded and fixed properly, or only the symptoms.
- Check the reopen rate and the defects that escaped to production.
- Check whether fixed defects were added to the regression suite.
- Look for process gaps: missing reviews, incomplete coverage, weak test data or poor environment.

### 3. How Repository Data Supports Test Design

- Defect-prone modules receive more tests and depth (defect clustering).
- Frequent defect types guide technique selection, e.g., many boundary defects lead to more BVA tests.
- Past failures become regression test cases.
- Severity and impact data support risk-based priority.
- Escaped defects reveal missing scenarios to add to test design.

### 4. Identifying Defect Patterns

Patterns are found through classification and analysis techniques such as Pareto charts, trend charts, fishbone diagrams and defect density comparisons.

Here, input validation is a recurring pattern and calculation/integration defects are growing, so those areas need targeted tests and preventive action.

## **5. Improvements to Prevent Recurring Defects**

- Mandatory root cause and phase-injected fields in every defect record.
- Periodic causal analysis meetings with preventive action plans.
- Add each fixed defect to automated regression tests.
- Update coding standards, review checklists and training on common error types.
- Review the repository before each release planning and before test design.
- Track metrics: defect recurrence rate, defect removal efficiency and escaped defects.

### Question 13

A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

#### 1. Introduction

The learning system fails during peak times (for example, exam registration or assignment deadline) because only functional tests were done with a few users. Non-functional requirements like load and scalability were not planned or tested.

#### 2. Identifying Gaps Using Testing Principles

- **Exhaustive testing is impossible:** but risk analysis should have identified peak load as a high-risk scenario.
- **Early testing:** performance requirements (users, response time) were not defined or reviewed early.
- **Context dependent:** an educational platform with thousands of simultaneous users needs scalability testing.
- **Defect clustering:** failures concentrate in login, quiz submission and video streaming.
- **Testing shows presence of defects:** passing functional tests does not show it will handle load.
- Gaps: no load, stress, endurance or spike tests; unrealistic test data and environment; no monitoring.

#### 3. Impact of Test Planning and Coverage on Performance

- A test plan states performance goals (e.g., 5,000 concurrent users, response time under 3 seconds, error rate below 1%).
- Coverage includes not only code and requirements but also load profiles, user scenarios and environment configurations.
- Without planning and coverage, bottlenecks (database queries, memory, thread pools, network) remain hidden until production.

#### **4. Better Test Strategies and Reliability**

Example: in a load test with JMeter at 2,500 users, the database connection pool is exhausted and login time reaches 20 seconds. After increasing the pool and adding query indexes, retest shows 2 seconds at 5,000 users.

#### **5. Enhancements to Avoid Such Failures**

- Define performance requirements and acceptance criteria early.
- Build a production-like test environment with realistic data.
- Use JMeter, Gatling or LoadRunner for load testing and run them in CI for each major release.
- Use monitoring and APM tools (Grafana, New Relic) to find bottlenecks.
- Plan capacity: caching, CDN, auto-scaling and graceful error handling.
- Prioritise critical flows by risk and perform failover and recovery testing.

## Question 14

A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

### 1. Introduction

Testing should not begin only after coding. Following the principle of early testing, testers should take part from the requirements stage onward, as in the V-model, where each development phase has a matching test activity.

### 2. Impact of Early Involvement on Quality

- Defects in requirements and design are found when fixing costs are lowest; cost of a defect grows greatly in later phases.
- Requirements become clearer and testable, reducing misunderstanding.
- Test planning and automation preparation run in parallel with development, shortening the schedule.
- Risk areas are identified early and tracked.
- Better collaboration reduces blame and rework.

### 3. Practical Example

In an online food-ordering project, a tester reviewing the requirement "Apply discount for orders above Rs. 500" asks whether it means above or equal to 500, and whether it applies before or after delivery charges. The ambiguity is clarified before coding. Without this, the developer would have implemented it wrongly and the defect would have been found during system testing, requiring changes in code, test cases and documentation.

### 4. Changes in Team Collaboration Practices

- Include testers in requirement, design and sprint planning meetings (shift-left).
- Adopt Agile/DevOps practices: daily stand-ups, three amigos sessions (business, developer, tester) and shared definition of done.
- Use common tools (Jira, Confluence) and share test plans and results.

- Pair testers and developers for unit tests and test-driven or behaviour-driven development.
- Hold joint defect triage and retrospectives for continuous improvement.

## Question 15

A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

### 1. Introduction

Inconsistent behaviour is a sign that only some inputs, paths or conditions were tested. Test coverage measures how much of the requirements or code has been exercised by tests and is an indicator of testing completeness.

### 2. Identifying Coverage Gaps Using Testing Principles

- Exhaustive testing is impossible, so coverage must be chosen by risk, but critical paths must not be left out.
- Compare the requirement traceability matrix with executed tests to find untested requirements.
- Use a code coverage tool (JaCoCo, Coverage.py) to find unexecuted statements, branches and conditions.
- Check input domain coverage: boundaries, invalid values and combinations.
- Check defect history: failures in areas with few tests indicate gaps.
- Review for the pesticide paradox: old tests no longer reach new code.

### 3. How Proper Coverage Ensures Reliability

- More requirements, paths and data classes are verified, so fewer hidden defects remain.
- Untested branches (such as error handling) are executed and fixed before release.
- Coverage metrics give objective evidence of testing completeness and release risk.
- Changes can be regression tested confidently.

### 4. Improving Test Case Design (Illustration)

Function: grade(marks): 90 and above = A, 75-89 = B, 50-74 = C, below 50 = Fail. Suppose the team tested only 95, 80 and 60.

The added tests would reveal, for example, that marks = 75 gives C because the developer wrote  $> 75$ .

## 5. Techniques for Better Validation

- Black-box: equivalence partitioning, BVA, decision tables, state-transition and cause-effect graphing.
- White-box: statement, branch, condition and path coverage.
- Pairwise (combinatorial) testing for many input combinations.
- Review and traceability audits, mutation testing to assess test quality.
- Exploratory and error-guessing tests for scenarios not in scripts.
- Set coverage targets (e.g., 100% requirement coverage, 80%+ branch coverage) as exit criteria.

## Question 16

A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

### 1. Introduction

In a logistics application (shipment booking, tracking, route planning) a poorly written report such as "tracking not working" delays diagnosis. Applying the testing concepts of error, defect and failure and using a defect life cycle makes reporting clear and fast.

### 2. Applying Testing Concepts to Reporting

- Report the failure (what was observed) with expected versus actual behaviour.
- State the suspected defect location (module, API or screen) only if known, without assuming the cause.
- Classify by type, severity and priority.
- Follow the defect life cycle: New, Assigned, Open, Fixed, Retest, Closed or Reopened.

### 3. How Clear Definitions Help Tracking

- A consistent meaning of defect, failure and severity makes counts and trends reliable.
- Each defect has a unique ID and status, so progress and ownership are visible.
- Duplicates are easily detected, and metrics like defect density and aging can be computed.
- Severity versus priority definitions let managers schedule fixes correctly.

### 4. Structured Reports Enhance Communication

The developer can reproduce the issue immediately, so identification and fixing take less time and there are fewer clarification messages.

### 5. Improvements in Reporting Standards

- Adopt a standard template (based on IEEE 829 / ISTQB) with mandatory fields.
- Use a tracking tool (Jira, Bugzilla) with workflow and notifications.
- Give guidelines for titles, steps and severity and priority definitions.

- Review reports for completeness before submission and hold daily triage.
- Track metrics such as time to report, time to fix and reopened defects.

## Question 17

A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

### 1. Introduction

When each module team tests in its own way, coverage, documentation and defect handling differ, so quality varies. A process-based approach defines one common testing process that all modules follow.

### 2. Process-Based Testing Approach

- **Define the process:** test planning, analysis and design, implementation, execution, reporting and closure, based on ISO/IEC/IEEE 29119 and ISTQB.
- **Standard artifacts:** common templates for test plan, test cases, defect reports and summary reports.
- **Entry and exit criteria:** same conditions to start and finish testing for every module.
- **Roles and responsibilities:** clear owners for each activity.
- **Process improvement:** use models such as TMMi or CMMI to assess and improve maturity.

### 3. How Standardised Processes Improve Quality

- Same minimum coverage and techniques for every module reduce gaps.
- Results and metrics are comparable across modules.
- Repeatable steps reduce dependence on individual skill.
- New members learn faster and handover is easier.
- Reviews and audits can be applied uniformly, leading to continuous improvement.

### 4. Example

A project has Billing, Inventory and Reports modules. Before: Billing has detailed cases, Inventory has only an informal checklist and Reports is tested at the end. After: all three follow the same steps with a template for test cases, BVA and equivalence partitioning mandatory for input fields, exit criteria of 100% requirement coverage and no open critical

defects, and the same defect template. Defect leakage across modules becomes similar, and the test manager can compare module quality using the same metrics.

## **5. Methods to Ensure Consistency**

- Publish a test strategy and guidelines for the organisation.
- Use one test management and defect tool for all modules.
- Peer review of test cases and checklists-based audits.
- Common automation framework and coding standards for scripts.
- Regular process audits, retrospectives and training.
- Monitor metrics (coverage, defect density, defect removal efficiency) against agreed targets.

## Question 18

A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

### 1. Introduction

Edge cases are inputs or conditions at the extreme limits of the operating range (minimum, maximum, empty, just inside or outside a limit). Defects commonly cluster at these boundaries, but they are often skipped when testing is informal.

### 2. Why Edge Cases Were Missed

- Exhaustive testing is impossible and the team did not use a technique to select important values.
- Requirements did not specify limits clearly, so testers did not know the boundaries.
- Tests used typical (nominal) values only.
- Testing started late and was cut short because of schedule pressure.
- Pesticide paradox: the same test set was reused without review.
- No test design review or coverage measurement.

### 3. How Boundary Testing Helps

Boundary Value Analysis (BVA) tests values at and around the edges of each equivalence partition, because developers commonly make off-by-one errors with  $<$ ,  $<=$ ,  $>$  and  $>=$ . With few test cases it covers the most defect-prone points.

### 4. Including Edge Cases in Testing (Illustration)

Example: an age field accepts 18 to 60 for registration.

More edge cases: empty input, very long strings, special characters, null, zero, negative numbers, leap-year dates (29 Feb), end-of-month, time zones, maximum file size and maximum number of items in a list.

### 5. Improvements in Test Design

- Define clear input ranges in requirements and review them with testers.

- Apply BVA with equivalence partitioning and robust BVA (values beyond limits) as a standard in design.
- Add negative and error-handling tests and state-transition tests for limit states.
- Use checklists of common edge cases and exploratory sessions.
- Measure coverage, review test cases and update tests every release.
- Add production edge-case defects to regression and consider fuzz testing.

## **Question 19**

A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

### **1. Introduction**

Without proper documentation, defects are lost in emails and chats, duplicates arise, and no one knows their status. A defect repository provides a central, structured store with a defined life cycle for all defects.

### **2. Applying Defect Repository Concepts**

- Introduce a defect tracking tool (Jira, Bugzilla, Azure DevOps) as the single repository.
- Define the defect record: ID, title, description, module, environment, version, steps to reproduce, expected and actual result, severity, priority, reporter, assignee, status, root cause, fix details and attachments.
- Define the life cycle: New, Assigned, Open, Fixed, Retest, Closed, Reopened, Deferred or Rejected.
- Control access, versioning and audit history of changes.

### **3. How Documentation Improves Defect Management**

- Developers can reproduce and fix faster because the information is complete.
- Ownership and status are visible, so defects are not forgotten.
- Duplicates are found and avoided by searching the repository.
- Management can make release decisions using open defect counts and severity.
- Knowledge is retained for future projects and audits.

### **4. Improvements in Documentation Practices**

- Use a mandatory standard template and field definitions for severity and priority.
- Require reproducible steps, evidence and environment details before a defect is accepted.
- Search for duplicates before logging; link related defects and test cases.

- Update status and fix details promptly and record root cause on closure.
- Review reports in triage meetings and train team members on reporting.
- Create periodic dashboards and share lessons learned.

## Question 20

A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves

### 1. Introduction

Repeated failures in transaction processing (deposit, withdrawal, transfer, bill payment) show that the testing practice is weak: incomplete cases, no regression, and no focus on high-risk areas. The testing principles guide a redesigned approach.

### 2. Redesigned Testing Approach Based on Principles

Process: plan, design, execute, report, regression, and measure, with exit criteria such as no open critical or major defects.

### 3. How Structured Testing Improves Quality

- Test cases cover valid, invalid and boundary values and all transaction states.
- ACID properties (atomicity, consistency, isolation, durability) are explicitly verified.
- Failures are traceable to requirements and repeatable.
- Automation of regression prevents recurrence of earlier failures.
- Metrics help to decide release readiness.

### 4. Illustrative Examples

- **Atomicity:** transfer Rs. 1,000 from A to B; simulate failure after debit and before credit. Expected: whole transaction rolled back, A's balance unchanged.
- **Boundary:** daily limit Rs. 1,00,000; test 99,999, 1,00,000 and 1,00,001.
- **Concurrency:** two simultaneous withdrawals of Rs. 8,000 from an account with Rs. 10,000. Expected: one succeeds, the other fails; the balance never becomes negative.
- **Duplicate submission:** click Pay twice quickly. Expected: only one debit.

### 5. Techniques to Enhance Defect Detection

- Boundary value analysis, equivalence partitioning, decision tables and state-transition testing.
- Database testing and reconciliation of ledger totals against transactions.
- Concurrency, performance, security and failure-recovery testing.

- Automated regression and API testing (Postman, REST Assured, Selenium) in CI.
- Peer reviews and static code analysis; exploratory testing for unusual cases.
- Defect repository analysis for pattern detection and prevention.

## 6. Conclusion

A principle-based, structured redesign makes testing of transactions planned, risk-focused and repeatable, so failures are detected earlier and the banking system becomes more reliable and trustworthy.

### RUBRICS FOR CALCULATION:

| Criteria                  | Weightage | Level 4 – Exemplary                                         | Level 3 – Proficient                    | Level 2 – Developing                         | Level 1 – Beginning                      |
|---------------------------|-----------|-------------------------------------------------------------|-----------------------------------------|----------------------------------------------|------------------------------------------|
| Understanding of Concepts | 25%       | Demonstrates thorough, accurate understanding of concepts   | Good understanding with minor gaps      | Basic understanding with some misconceptions | Poor understanding; major misconceptions |
| Explanation               | 25%       | Detailed, well-explained answers with relevant examples     | Clear explanation with adequate details | Limited explanation; lacks depth             | Poor or unclear explanation              |
| Application               | 20%       | Effectively applies concepts with strong analytical ability | Applies concepts with minor errors      | Limited application with weak analysis       | No application or incorrect analysis     |
| Organization              | 15%       | Well-structured answers with logical flow and coherence     | Organized with minor issues in flow     | Basic structure, lacks clarity               | Poor organization, incoherent answers    |
| Accuracy                  | 10%       | Completely accurate and covers all required points          | Mostly accurate with minor omissions    | Partially correct with missing points        | Incorrect answers with major gaps        |
| Clarity                   | 5%        | Clear, neat, professional presentation                      | Understandable with minor issues        | Limited clarity, readability issues          | Poorly written, difficult to understand  |